

Schule
Jahrgangsstufe
Schuljahr

Facharbeit

Einfluss von Programmiersprachen auf die Ressourceneffizienz von Computerspielen

Eine Analyse am Beispiel von RollerCoaster Tycoon und
einem Vergleich von Assembly x86-64, C++ und JavaScript
anhand eines Minispiels

vorgelegt von:

Name

Fachlehrer/in:

Lehrer

Fach:

Medienkompetenz

Abgabedatum:

Datum

Inhaltsverzeichnis

1 Einleitung.....	3
2 Theoretischer Hauptteil.....	4
2.1 Ressourceneffizienz in der Softwareentwicklung.....	4
2.1.1 Definitionen.....	4
2.1.1.1 CPU-Auslastung.....	4
2.1.1.2 Arbeitsspeicher-Auslastung.....	4
2.1.1.3 Speicherplatz.....	4
2.1.1.4 Ladezeiten.....	5
2.1.1.5 Ressourceneffizienz.....	5
2.1.2 Einflussfaktoren auf die Ressourceneffizienz von Programmiersprachen.....	5
2.2 Überblick über die Programmiersprachen.....	7
2.2.1 Assembly (x86_64).....	7
2.2.2 C++.....	9
2.2.3 Javascript.....	9
3 Fallstudie: RollerCoaster Tycoon 1.....	11
3.1 Entstehung und historischer Kontext.....	11
3.2 Analyse der Ressourcennutzung.....	12
3.2.1 Spielprinzip und Mechaniken.....	12
3.2.2 Warum war bzw. musste RCT so effizient sein?.....	12
3.3 Bedeutung für moderne Spielentwicklung.....	13
4 Vergleichsprojekt: Minispiel in Assembly (x86_64), C++ und Javascript.....	14
4.1 Aufbau und Spielmechaniken.....	14
4.2 Gemeinsamkeiten und Unterschiede in der Implementierung der Spielmechaniken in den verschiedenen Sprachen.....	15
4.2.1 x86_64 Assembly.....	15
4.2.2 C++.....	16
4.2.3 Javascript / Node.js.....	16
4.3 Messung und Vergleich.....	16
5 Fazit.....	19
6 Bibliografie.....	I
6.1 Quellenverzeichnis.....	I
6.2 Abbildungsverzeichnis.....	II
7 Anhang.....	II
8 Eidesstattliche Erklärung.....	III

1 Einleitung

Ein Snake-Spiel, so kompakt, dass es in einen QR-Code passt – kaum zu glauben! Obwohl ein QR-Code maximal 2956 Bytes speichern kann¹, haben Entwickler ein vollständiges Spiel erschaffen, das gerade einmal 54 Bytes groß ist. Möglich wurde das durch die Verwendung von x86-Assembly². Als ich dieses Projekt auf GitHub entdeckte, war ich sofort fasziniert und fragte mich: Wie genau beeinflusst die Wahl der Programmiersprache die Größe eines Programms?

Diese Frage stellte ich mir auch beim Rückblick auf meine Kindheit, als ich begeistert *RollerCoaster Tycoon* spielte, eine Freizeitpark-Simulation aus den späten 1990er Jahren. Trotz begrenzter Hardware-Ressourcen simulierte das Spiel komplexe Abläufe wie Besucherströme, Wartungspersonal und Achterbahnen erstaunlich flüssig. Auch dieses Spiel wurde in Assembly geschrieben³. Auch hier stellte sich mir die Frage: Welchen Einfluss hat die Programmiersprache auf die Ressourceneffizienz?

In dieser Facharbeit untersuche ich den Zusammenhang zwischen Programmiersprache und Ressourceneffizienz anhand zweier Beispiele: *RollerCoaster Tycoon* als historisches Referenzprojekt und einem eigens entwickelten Spiel, das in drei verschiedenen Sprachen, Assembly, C++ und JavaScript, umgesetzt wurde. Ziel ist es, die Auswirkungen der Sprachwahl auf Speicherbedarf, Ausführungsgeschwindigkeit und Komplexität zu analysieren. Ich stelle die Hypothese auf: Je näher eine Programmiersprache an der Maschinenebene arbeitet, desto effizienter ist das resultierende Programm.

1 Denso Wave Incorporated: *Information capacity and versions of the QR Code*. Qrcode.com, verfügbar unter: <https://www.qrcode.com/en/about/version.html>, eingesehen am 31.10.2025 13:24

2 donno2048: *54-byte snake game*. Github.com, verfügbar unter: <https://github.com/donno2048/snake>, eingesehen am 25.02.2025 00:20

3 Sawyer, Chris: *...about Chris Sawyer & Game Development*. chrissawyergames.com, verfügbar unter: <https://www.chrissawyergames.com/faq3.htm>, eingesehen am 31.10.2025 15:20

2 Theoretischer Hauptteil

2.1 Ressourceneffizienz in der Softwareentwicklung

2.1.1 Definitionen

2.1.1.1 CPU-Auslastung

Die Prozessorauslastung (auch CPU-Auslastung) beschreibt, welchen Anteil seiner Arbeitszeit einer oder mehrere der Hauptprozessoren eines Computers tatsächlich produktive Aufgaben bearbeitet und wird in üblicherweise prozentual angegeben⁴.

2.1.1.2 Arbeitsspeicher-Auslastung

*„Der Arbeitsspeicher, kurz RAM (Random Access Memory), ist [...] für die vorübergehende Speicherung von Daten zuständig, während Programme ausgeführt werden“.*⁵

Die Arbeitsspeicher-Auslastung beschreibt, wie viel des verfügbaren Arbeitsspeichers absolut oder prozentual genutzt wird.

2.1.1.3 Speicherplatz

*„Die Speicherkapazität benennt die maximale Datenmenge, die sich in einem Datenspeicher speichern lässt.“*⁶

In dieser wissenschaftlichen Arbeit bezeichnet „belegter Speicherplatz“ diejenige Menge an nicht-flüchtigem Speicher (Festplatte oder SSD), die von einem Programm und seinen zur Ausführung erforderlichen Dateien (z.B Bibliotheken) tatsächlich belegt wird.

4 Handermann, Sören und Hery-Moßmann, Nicole: *Was ist die CPU-Last?*. Chip.de, 25.08.2023, verfügbar unter: https://www.chip.de/ratgeber/pc-mac-zubehoer/was-ist-die-cpu-last_38da44b0-d66e-40ea-ade4-7da337b45d47.html, eingesehen am 11.01.2026 20:39

5 Feldten, Jan Philipp / DeLSt GmbH - Deutsches eLearning Studieninstitut: *Arbeitsspeicher (RAM)*. Delst.de, verfügbar unter: <https://www.delst.de/de/lexikon/arbeitsspeicher-ram/>, eingesehen am 11.01.2026 21:08

6 Luber, Stefan und Jürgen Ehneß: *Was ist Speicherkapazität?*. Storage-insider.de, 21.01.2020, verfügbar unter: <https://www.storage-insider.de/was-ist-speicherkapazitaet-a-885599/>, eingesehen am 25.02.2025 00:50

2.1.1.4 Ladezeiten

„Unter Ladezeit versteht man die Zeitspanne, die ein Element benötigt, um korrekt auf dem Monitor des Betrachters dargestellt zu werden.“⁷

2.1.1.5 Ressourceneffizienz

„Ressourceneffizienz bezieht sich auf die effiziente Nutzung und den sparsamen Einsatz von natürlichen Ressourcen, um Produkte und Dienstleistungen zu erstellen. Das Ziel ist, den Verbrauch natürlicher Ressourcen zu minimieren und dabei gleichzeitig den wirtschaftlichen Nutzen zu maximieren.“⁸

In dieser wissenschaftlichen Arbeit bezeichnet Ressourceneffizienz die effiziente und sparsame Verwendung von Hardwarekomponenten sowie die Minimierung von z.B. Ladezeiten und belegtem Speicherplatz bei gleichzeitiger Maximierung der Nutzungsqualität und Funktionalität der Software.

2.1.2 Einflussfaktoren auf die Ressourceneffizienz von Programmiersprachen

Um die Einflussfaktoren auf die Ressourceneffizienz zu bestimmen, müssen zuerst die Unterschiede zwischen verschiedenen Programmiersprachen betrachtet werden. Einer der Hauptunterschiede bei Programmiersprachen ist, ob die Sprache interpretiert oder kompiliert wird. Eine kompilierte Sprache wandelt den gesamten Code direkt in Maschinencode um, damit der Prozessor ihn ausführen kann. Damit hat der Entwickler mehr Kontrolle über Hardwareaspekte wie Speicherverwaltung und CPU-Nutzung, muss aber bei jeder Änderung das Programm mittels eines „Build“-Skripts erneut in Maschinencode umwandeln. Beispiele für rein kompilierte Sprachen sind C, C++ oder Rust.

Interpretierte Sprachen hingegen nutzen einen Interpreter, der das Programm Zeile für Zeile durchgeht und jeden Befehl ausführt. Damit sind interpretierte Sprachen oftmals flexibler und der Code ist plattformunabhängig. Beispiele für interpretierte Sprachen sind PHP, Python und JavaScript.⁹

⁷ Fuchs Media Solutions (Hrsg): *Ladezeit Begriffserklärung und Definition*. SEO-Analyse.com, verfügbar unter: <https://www.seo-analyse.com/seo-lexikon//ladezeit/>, eingesehen am 11.01.2026 21:41

⁸ Hulatt, Lily: *Ressourceneffizienz*. Studysmarter.de, 28.02.2024, verfügbar unter: <https://www.studysmarter.de/studium/bwl/makrooekonomie-studium/ressourceneffizienz/>, eingesehen am 13.01.2026 13:06

⁹ Hersel, Ralf: *Interpretiert versus Kompiliert*. Gnulinux.ch, 26.10.2021, verfügbar unter: <https://gnulinux.ch/interpretiert-versus-kompiliert>, eingesehen am 18.01 17:36

Im Vergleich zeigen sich kompilierte Sprachen oftmals als effizienter, stellt eine Studie von Universitäten aus Portugal fest. Dabei verbrauchten z.B. C oder C++ weniger Energie als interpretierte Sprachen. Dahingegen waren interpretierte Sprachen meist schneller und energieeffizienter.¹⁰

Ein weiterer Faktor ist das Speicherverwaltungsmodell. Java, Python oder Go nutzen Garbage-Collection um nicht-genutzten oder überflüssigen Speicher freizugeben. Garbage-Collection-Algorithmen können jedoch zu neuen Verzögerungen führen. Dahingegen wird Speicher in C oder C++ manuell zugewiesen und freigegeben. Dabei können potenziell Fehler oder Schwachstellen, was zu einer Erhöhung der Arbeitsspeicher-Auslastung führen kann.¹¹

Etwa 90% einer Software ist nicht vom eigentlichen Entwickler geschrieben, sondern besteht aus Software-Bibliotheken¹². Diese Bibliotheken sind ein weiterer Faktor, welcher auf die Ressourceneffizienz Einfluss nehmen kann. Der Optimierungsgrad der Bibliothek ist hier nun entscheidend. Stark optimierte Bibliotheken wie z.B. NumPy in Python, welche in C geschrieben wurde¹³, können die Effizienz einer Software verbessern. Auf der anderen Seite können ungenutzte Module oder zusätzliche Hintergrundprozesse die Effizienz deutlich negativ beeinflussen.

Die Hardwarenähe einer Programmiersprache beschreibt, wie direkt der geschriebene Code auf die physische Hardware zugreifen kann, bspw. CPU-Register, Speicheradressen oder Cache-Strukturen. Je geringer die Abstraktionsebene ist, desto weniger Zwischenschichten müssen zur Laufzeit aktiv sein. Das wirkt sich unmittelbar auf die Ressourceneffizienz aus, weil jeder zusätzliche Layer Rechenzeit, Energie und Speicher benötigt. Sprachen wie C und C++ gelten als besonders hardwarenah, weil sie nahezu ohne Laufzeitumgebung auskommen und direkt in Maschinencode übersetzt werden. Dadurch entstehen

10 Digital Marketing Team at Creative Programs & Systems: *Energy Efficiency in Programming Languages*. Cpsmi.com, 12.05.2025, verfügbar unter: <https://www.cpsmi.com/blog/energy-efficiency-in-programming-languages/>, eingesehen am 18.01.2026 17:59

11 Heller, Martin: *Garbage Collection erklärt*. Computerwoche.de, 25.09.2023, verfügbar unter: <https://www.computerwoche.de/article/2820549/garbage-collection-erklart.html>, eingesehen am 23.01.2026 18:18

12 Dörnenburg, Erik: *Die Krux mit den Software-Bibliotheken*. Connect-professional.de, 04.04.2023, verfügbar unter: <https://www.connect-professional.de/security/die-krux-mit-software-bibliotheken-203182.html>, eingesehen am 23.01.2026 19:37

13 NumPy Developers: *What is NumPy?*. NumPy.org, verfügbar unter: <https://numpy.org/doc/stable/user/whatnumpy.html>, eingesehen 25.02.2026 01:43

kaum Interpretations- oder Virtualisierungsprozesse, was zu sehr kurzen Ausführungszeiten und niedrigem Energieverbrauch führt.

Im Gegensatz dazu benötigen höhere Programmiersprachen mehr Schichten, wie Garbage-Collection, Bytecode-Interpretation oder JIT-Kompilierung. Dadurch wird der Code lesbarer und besser verwaltbarer.¹⁴

Die Unterschiede von Programmiersprachen haben direkten Einfluss auf die Ressourceneffizienz. Je näher die Programmiersprache an der Hardware ist, umso ressourceneffizienter kann sie sein. Höher abstrahierte Sprachen erleichtern die Entwicklung, sind aber ressourcenintensiver. Man tauscht Entwicklungseffizienz gegen Ressourceneffizienz. Entscheidend ist daher nicht allein die Wahl der Programmiersprache, sondern vor allem die Art und Weise, wie sie genutzt wird. In jeder Sprache lässt sich sowohl effizienter als auch ineffizienter Code schreiben.

2.2 Überblick über die Programmiersprachen

2.2.1 Assembly (x86_64)

„Assemblersprache ist eine Low-Level-Programmiersprache, die mnemonische Codes zur Darstellung von Maschinenbefehlen verwendet. Es handelt sich um eine für Menschen lesbare Form der Maschinensprache, die eine engere Übereinstimmung zwischen den Anweisungen, die von der Hardware des Computers verstanden werden, und den vom Programmierer geschriebenen Anweisungen bietet. In der Assemblersprache arbeiten Sie direkt mit der Architektur des Computers und können die Hardware auf einer feineren Ebene steuern als in höheren Sprachen.“¹⁵

Assemblersprachen zeichnen sich durch eine extrem hohe Hardwarenähe aus. Dadurch erhalten Entwickler*innen präzise Kontrolle über Register, Speicheradressen und den Ablauf einzelner CPU-Operationen. Geschriebener Code wird direkt in Maschinensprache übersetzt. Jeder Befehl entspricht einer Ma-

14 Rauch, Gedeon: *Was sind höhere Programmiersprachen?*. Datacenter-inside.de, 11.03.2022, verfügbar unter: <https://www.datacenter-insider.de/was-sind-hoehere-programmiersprachen-a-35361189a0d1b8c91cb7b8f209abab10/>, eingesehen am 25.01.2026 20:07

15 Lenovo Germany GmbH: *Assembly Language*. Lenovo.com, verfügbar unter: <https://www.lenovo.com/de/de/glossary/assembly-language/?orgRef=https%253A%252F%252Fwww.google.com%252F&srsId=AfmBOopkzKV1Vv-0FmC12zCIng3JSOrMS1U6d6B9QHmnAxKd1aoFnxU>, eingesehen 25.02.2026 02:16

schinencode-Anweisung, deswegen ist Assembly oftmals für Menschen schwerer lesbar (siehe z.B. Abbildung 1).

Der Begriff „Assemblersprache“ bezieht sich nicht auf eine einzige Sprache, sondern auf eine ganze Familie an Sprachen. Sie unterscheiden sich im Grad der Abstraktion, in der Semantik oder in der unterstützten Architektur.¹⁶

In dieser Facharbeit wird die Assemblersprache x86-64 betrachtet, die auf modernen 64-Bit-Prozessoren von Intel und AMD eingesetzt wird. Sie stellt eine Weiterentwicklung der klassischen x86-Architektur dar und bietet erweiterte Register, zusätzliche Instruktionssätze und optimierte Speicheradressierungsmodi.

17

```
section .data
    msg db 'Hello, World', 0      ; definiert einen nullterminierten
    string, indem jeder char in einem byte gespeichert ist

section .text ; ausführbaren code initiieren
    global _start

_start:
    ; print hello world
    mov rax, 1      ; systemaufruf für sys_write (1)
    mov rdi, 1
    mov rsi, msg    ; adresse des strings
    mov rdx, 13     ; länge des strings (bytes)
    syscall         ; systemaufruf initiieren

    ; exit
    mov rax, 60     ; systemaufruf zum verlassen (sys_exit (60))
    xor rdi, rdi    ; exit code 0
    syscall
```

Abbildung 1: Hello-World Code in x86_64 Assembly

¹⁶ ebda. Lenovo Germany GmbH: *Assembly Language*.

¹⁷ TU Dresden, Professur Betriebssysteme: *Assembler Crash Course (x86_64)*. Os.inf.tu-dresden.de, verfügbar unter: <https://os.inf.tu-dresden.de/Studium/OSC/SS2022/tasks/task4/assembler.php>, eingesehen am 25.02.2026 02:37

2.2.2 C++

C++ ist eine von der ISO genormte Programmiersprache. Bjarne Stroustrup entwickelte C++ 1983 als Erweiterung der Programmiersprache C. Sie ermöglicht sowohl die Programmierung auf hohem Abstraktionsniveau als auch eine effiziente und maschinennahe Programmierung.¹⁸

Durch die direkte Übersetzung in Maschinencode und den geringen Laufzeit-Overhead bietet C++ eine sehr hohe Ausführungsleistung und Ressourceneffizienz. Die Sprache erlaubt manuelle Speicherverwaltung, wodurch Entwickler präzise Kontrolle über Heap- und Stack-Nutzung erhalten. Gleichzeitig unterstützt C++ moderne Sprachfeatures wie Objektorientierung, Templates und funktionale Elemente, was die Entwicklung komplexer Systeme erleichtert, ohne die Effizienz der Ausführung wesentlich zu beeinträchtigen.¹⁹

C++ wird sehr vielfältig in Bereichen wie Systemsoftware, Spielentwicklung, eingebettete Systeme, wissenschaftlichen Berechnungen und Hochleistungsanwendungen eingesetzt. Unter anderem wird C++ auch in der Unreal Engine verwendet, da es eine optimale Balance zwischen Performance, Flexibilität und Kontrolle bietet.²⁰

```
#include <iostream> // standard-bibliothek für z.B. Ausgabefunktionen

int main() { // Hauptfunktion
    std::cout << "Hello World"; // cout = character output, gibt string
    aus
    return 0; // fehlerfreies beenden
}
```

Abbildung 2: Hello-World Code in C++

2.2.3 Javascript

„JavaScript (JS) ist eine leichtgewichtige, interpretierte (oder Just-in-Time compilierte) Programmiersprache mit First-Class Funktionen. Obwohl es am bekanntesten als Skriptsprache für Webseiten ist, wird es auch in Nicht-Browser-Umgebungen genutzt, wie beispielsweise Node.js, Apache CouchDB und Adobe

18 Barney, Nick: C++. ComputerWeekly.de, 05.052024, verfügbar unter: <https://www.computerweekly.com/de/definition/C>, eingesehen 01.02 20:44

19 ebda. Barney, Nick: C++

20 ebda. Barney, Nick: C++

*Acrobat. JavaScript ist eine prototypenbasierte, automatisch speicherbereinigende, dynamische Sprache, die mehrere Paradigmen wie imperatives, funktionales und objektorientiertes Programmieren unterstützt.*²¹

Javascript ähnelt C-Abkömmlingen. Trotz der Namensähnlichkeit hat Javascript wenig Gemeinsamkeiten mit der Programmiersprache Java. Javascript ist als ECMAScript standardisiert.²²

Im Gegensatz zu C++ oder Assembly läuft JavaScript nicht direkt auf der Hardware, sondern innerhalb einer Laufzeitumgebung wie der V8-Engine (Chrome, Node.js) oder SpiderMonkey (Firefox). Für diese wissenschaftliche Arbeit wird JavaScript in Kombination mit Node.js betrachtet. Die Nutzung von Node.js ermöglicht eine kontrollierte, reproduzierbare und mit anderen Programmiersprachen vergleichbare Ausführungsumgebung, da der Code nicht im Browser, sondern in einer stabilen, serverseitigen Laufzeitumgebung ausgeführt wird. Dies erleichtert die Analyse der Ressourceneffizienz und stellt sicher, dass Unterschiede zwischen den Programmiersprachen nicht durch unterschiedliche Plattformen oder Browser-Implementierungen verfälscht werden.²³

```
console.log(„Hello World“)
```

Abbildung 3: Hello-World Code in Javascript

21 Mozilla Foundation: *JavaScript*. Developer.mozilla.org, verfügbar unter: <https://developer.mozilla.org/de/docs/Web/JavaScript>, eingesehen 25.02.2026 03:31

22 ebda. Mozilla Foundation: *JavaScript*

23 IONOS Redaktion: *Node.js-Einführung: Die JavaScript-Laufzeitumgebung im Überblick*. Ionos.at, 11.09.2025, verfügbar unter: <https://www.ionos.at/digitalguide/websites/web-entwicklung/einfuehrung-in-nodejs/>, eingesehen am 02.02 14:32

3 Fallstudie: RollerCoaster Tycoon 1

3.1 Entstehung und historischer Kontext

„Unverkäuflich“ wurde Chris Sawyer von Experten der Videospielindustrie während der Entwicklung von *RollerCoaster Tycoon* (RCT) gesagt²⁴. Schlussendlich wurde es das meistverkaufte Videospiel auf dem PC im Jahr 1999.²⁵

Der Vorgänger *Transport Tycoon* (TT) war Chris Sawyer erster richtiger Erfolg. Es ist eine Verkehrs- und Wirtschaftssimulation von 1994, die in isometrischer Grafik dargestellt wird. Ziel des Spiels war es, ein erfolgreiches Transportunternehmen aufzubauen.²⁶ Programmiert wurde es alleine von Chris Sawyer und lediglich die finalen Grafiken und die Musik wurden von anderen Personen erstellt.²⁷

Während der Entwicklung des Nachfolgers *Transport Tycoon 2* entdeckte Chris Sawyer im Frühling 1996 seine Faszination für Vergnügungsparks und legte deshalb den Programmcode immer mehr von Zügen auf Achterbahnen um²⁸. 1999 erschien dann *RollerCoaster Tycoon* und 2003 *RollerCoaster Tycoon 2* für den PC. Auch dieses Spiel wurde größtenteils im Alleingang von Sawyer programmiert.²⁹

24 Red Phoenix: *Interview with Chris Sawyer*. RCT Station Archive (archiviert): Internet Archive: Wayback Machine, 21.12.2013, verfügbar unter:

<https://web.archive.org/web/20131221171913/http://www.rctstationarchive.com/interviews/chrissawyer.shtml>, eingesehen am 04.02.2026 21:45

25 IGN STAFF: *PC Data Top Selling Games of 1999*. IGN.com, 22.06.2012, verfügbar unter: <https://www.ign.com/articles/2000/01/25/pc-data-top-selling-games-of-1999>, eingesehen am 05.02.2026 17:43

26 Sawyer, Chris (Entw.): *Transport Tycoon* [Computerspiel]. MicroProse, 1994. Gespielt am 06.02.2026 13:12

27 Fröhlich, Petra: *Rollercoaster Tycoon 4: Chris Sawyer und Rollercoaster Tycoon 4*. pcgames.de, 27.06.2009, verfügbar unter: <https://www.pcgames.de/Rollercoaster-Tycoon-4-Spiel-30756/Specials/Rollercoaster-Tycoon-4-Chris-Sawyer-und-Rollercoaster-Tycoon-4-688413/>, eingesehen am 05.02.2026 18:29

28 CarlyF (Autor), atari-club (Herausgeber): *Interview with RollerCoaster Tycoon's Creator, Chris Sawyer*. Medium.com, 27.03.2024, verfügbar unter: <https://medium.com/atari-club/interview-with-rollercoaster-tycoons-creator-chris-sawyer-684a0efb0f13>, eingesehen am 05.02.2026 18:42

29 ebda. Fröhlich, Petra: *Rollercoaster Tycoon 4: Chris Sawyer und Rollercoaster Tycoon 4*.

3.2 Analyse der Ressourcennutzung

3.2.1 Spielprinzip und Mechaniken

RollerCoaster Tycoon ist ein Aufbauspiel mit Aspekten einer Wirtschaftssimulation rund um den Aufbau von Vergnügungsparks. Am Anfang wählt der Spieler zwischen verschiedenen Szenarien. Diese unterscheiden sich in der Landschaft und geben jeweils bestimmte Aufgaben vor. Beispielsweise startet der Spieler im ersten Szenario „Forest Frontiers“ nur mit einer leicht bewaldeten, grünen Fläche und einem Kassenhäuschen. Die Herausforderung ist „*To have at least 250 guests in your park at the end of October, Year 1, with a park rating of at least 600*“. Der Spieler erhält ein Startbudget von 10.000 \$ und muss damit schrittweise Achterbahnen, Karussells, Restaurants und andere Attraktionen bauen.³⁰ Währenddessen muss der Spieler auf die Bewertung des Parks, auf die Gestaltung, Marketingkampagnen, Personal und Reparaturen an Attraktionen achten. Man kann die Zufriedenheit oder die Gedanken jedes einzelnen Besucher betrachten und danach den Park entwickeln. Beim Hinzufügen neuer Attraktionen, kann man entweder vorgefertigte Fahrgeschäfte wählen oder eigenhändig konstruiert und gestaltet werden. (siehe Abbildung 4).



Abbildung 4: Beispiel einer von Spielern gebauten Achterbahn

3.2.2 Warum war bzw. musste RCT so effizient sein?

„At the time there was no option other than to use machine code for RollerCoaster Tycoon.“³¹

Chris Sawyer verfügte nicht über die technischen Ressourcen, um ein ressourcenintensives Spiel in einer höheren Programmiersprache zu entwickeln. Die Computer der damaligen Zeit hätten die Komplexität der Simulation nicht bewältigen können.

³⁰ Sawyer, Chris (Entw.): *RollerCoaster Tycoon* [Computerspiel]. Hasbro Interactive, 1999. Gespielt am 08.02.2026 12:13

³¹ ebda. CarlyF (Autor), atari-club (Herausgeber): *Interview with RollerCoaster Tycoon's Creator, Chris Sawyer*.

RollerCoaster Tycoon wurde zu 99% in x86-Assembler/Maschinencode geschrieben, lediglich 1% der Funktionen, hauptsächlich für die Schnittstelle zu Windows und DirectX, wurden in C programmiert. Die Hardware der späten 1990er-Jahre (typischerweise PCs mit 90–200 MHz Prozessoren und 16–32 MB RAM) wäre mit der Simulation tausender Gäste, komplexer Achterbahn-Physik und detaillierter Animationen in einer Hochsprache überfordert gewesen.³²

Sawyer entwickelte die gesamte Spielengine, inklusive der Physik-Engine, selbst. Dies ermöglichte eine perfekte Anpassung an die Anforderungen des Spiels.

„It wasn't just small chunks of code that benefited from being very efficient machine code either — because of the number of objects the game had to cope with, virtually every bit of code involved with object handling needed to be ultra-efficient or the inefficiencies quickly multiplied up with a busy park.“³³

Durch diesen Ansatz entstanden keine überflüssigen oder ungenutzten Funktionen, die die Ressourceneffizienz beeinträchtigt hätten.

Die isometrische Grafikengine wurde ebenfalls vollständig in Assembler entwickelt und verzichtete auf zwischengeschaltete APIs. Sawyer nutzte einen 8-Bit-„Byte-per-Pixel“-Modus mit fester Palette, der schneller zu verarbeiten war als vollständige Palettenmodi, und implementierte spezialisierte Algorithmen für die Echtzeit-Farbverarbeitung von Objekten.³⁴

3.3 Bedeutung für moderne Spielentwicklung

RollerCoaster Tycoon zeigt, dass maßgeschneiderte Programmierung und ein tiefes Verständnis von Hardware, die Ressourceneffizienz von Software maßgeblich beeinträchtigen kann. Jedoch bedeutet diese Effizienz auch eine schlechtere Portabilität und Flexibilität. Mit der Weiterentwicklung von Hard- und Software kann es erneut viel Zeit kosten, die Software zu portieren. Es hat tat-

32 ebda. CarlyF (Autor), atari-club (Herausgeber): *Interview with RollerCoaster Tycoon's Creator, Chris Sawyer.*

33 ebda. CarlyF (Autor), atari-club (Herausgeber): *Interview with RollerCoaster Tycoon's Creator, Chris Sawyer.*

34 ebda. CarlyF (Autor), atari-club (Herausgeber): *Interview with RollerCoaster Tycoon's Creator, Chris Sawyer.*

sächlich länger gedauert, dass Spiel in C++ erneut zu schreiben, als das original Spiel.³⁵

Den Ansatz für moderne Simulationsspiele wie z.B. *Minecraft* legte teilweise *RollerCoaster Tycoon*. Die Fokussierung auf inhaltlichen und spielerischen Elementen statt Grafik und technischen Komponenten ist nicht nur bei *RCT* zuerkennen. Während zeitgenössische 3D-Titel visuell alterten, wirken die pixeligen isometrischen Grafiken von *RCT* auch mehr als 25 Jahre nach Release stilistisch konsistent.

4 Vergleichsprojekt: Minispiel in Assembly (x86_64), C++ und Javascript

4.1 Aufbau und Spielmechaniken

Das Minispiel ist inspiriert von dem „Dino-Spiel“, auch bekannt als „Chrome Dino“. Ziel des Spiels ist es mit der T-Rex-Spielfigur die Hindernisse zu überwinden und dadurch Punkte zu sammeln (siehe Abbildung 5). Das Minispiel dieser Facharbeit ist eine vor allem grafisch vereinfachte Version dieses Browser-spiels.

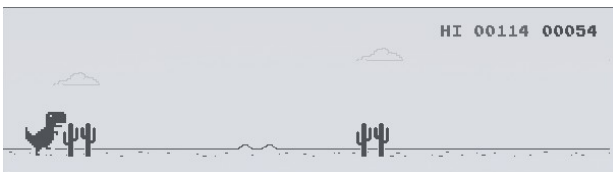


Abbildung 5: Screenshot aus Original Chrome-Dino Spiel

Das Spiel wird in allen drei Programmiersprachen in der Shell ausgeführt und gespielt. Es handelt sich um ein 15x60 Zeichen großes Spielfeld und um einen 2x2 großen Spieler. Nach dem

Start des Programms nähern sich Hindernisse von rechts nach links dem Spieler. Dieser kann nur durch das drücken von <SPACE> nach oben springen und damit ausweichen. Die Hindernisse können sechs verschiedene Formen annehmen und bestehen aus verschiedenen Zeichen wie z.B. '#' oder 'X' (siehe Abbildung 6). Durch das Überwinden von Hindernissen wird der Punktestand des Spielers erhöht.

35 ebda. CarlyF (Autor), atari-club (Herausgeber): *Interview with RollerCoaster Tycoon's Creator, Chris Sawyer*.

Berührt der Spieler ein Hindernis ist das Spiel vorbei und der Endpunktestand sowie der Rekord des Entwicklers wird angezeigt. Die Schnelligkeit des Spiels verhält sich proportional zum Punktestand.

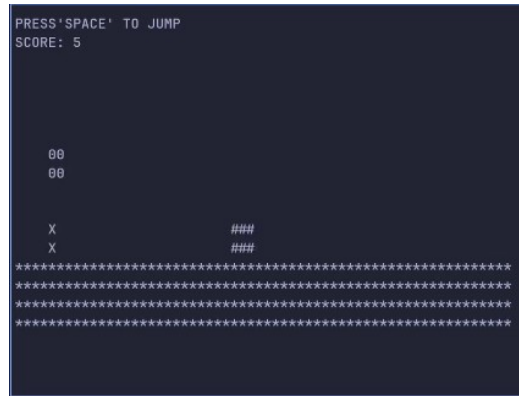


Abbildung 6: Praxisteil-Minispiel im Terminal

4.2 Gemeinsamkeiten und Unterschiede in der Implementierung der Spielmechaniken in den verschiedenen Sprachen

Alle drei Versionen wurden auf Arch Linux (x86_64) und Ubuntu mit dem Terminalemulator Kitty getestet.

Damit in einem Terminalemulator Nutzereingaben erkannt werden können und ungewohnte Eingaben nicht das Spielgeschehen stören, wird das Terminal am Anfang in allen Versionen in den ‚raw‘ – Modus gesetzt. Um das Terminal zu leeren oder um den Cursor zu verstecken werden ANSI-Escape-Codes genutzt. Nach dem Abschluss des Spiels wird das Terminal zurückgesetzt.

4.2.1 x86_64 Assembly

Die Assembly-Version ist im Intel-Syntax geschrieben. Am Anfang werden konstante Zahlen, wie die verschiedene Systemaufrufe („syscalls“), festgelegt. In der `.data` -Sektion werden bereits bekannte Variablen definiert. Hier sind z.B die Höhe und Breite des Spielfeldes festgelegt. Für Variablen, denen noch kein fester Wert zugewiesen wurde, wird in der `.bss` – Sektion Platz im Speicher reserviert. In der `.text` – Sektion werden alle benötigten Funktionen definiert und ausgeführt. In `_start` wird der Main-Loop initiiert. Dieser wiederholt sich alle 50.000.000 Nanosekunden (50ms). Dafür wird ein Sleep-Macro genutzt, um sich wiederholenden Code zu vermeiden. In `drawCanvas` wird über die Höhe und Breite iteriert, um ein Koordinatensystem zu erstellen. Dabei werden verschiedene Funktionen genutzt um zu prüfen, ob bspw. der Spieler an dieser Stelle gezeichnet werden soll. Die aktuellen Koordinaten werden mit den vorge-

geben Koordinaten des Spielers verglichen und ggf. wird der entsprechende Character in das Register *rax* geladen und mit der Funktion *print* ausgegeben. Die Hindernisse werden dynamisch während der Laufzeit erzeugt. Es gibt eine maximale Anzahl an Hindernissen, die gleichzeitig existieren können. Wenn ein Hindernis den linken Rand überquert, wird es gelöscht und ein Neues wird generiert und gezeichnet. Die Koordinaten für die jeweiligen Objekte im Spiel werden mit jeweils x- und y-Arrays gespeichert. Dabei setzt sich ein Punkt immer aus den Elementen am gleichen Index zusammen. Wenn der Spieler *<SPACE>* drückt, werden die y-Koordinaten des Spielers erst inkrementiert und anschließend dekrementiert, um einen realistischen Sprung zu simulieren. Abschließend wird das Spiel nach dem ‚GAME OVER‘ mit dem Systemaufruf 60 (exit) fehlerfrei beendet.

4.2.2 C++

Der Syntax und die Struktur ist in dieser Version deutlich lesbarer. Anfangs werden wieder Variablen und Konstanten mit entsprechenden Datentypen definiert. Der Spieler und die Hindernisse werden nun über Klassen und Objekte verwaltet. In der Klasse ‚*Player*‘ sind Attribute wie die Koordinaten oder das entsprechende Symbol und Funktionen wie z.B *jump()* definiert. In der *Obstacle* -Klasse sind Breite und Höhe der eines Hindernisses vorgegeben, außerdem gibt Hilfsfunktionen wie z.B *move()* oder *isVisible()*. Die *Obstacle* -Objekte werden in der *spawnObstacle()*- Funktion initiiert und in einem *std::vector* gespeichert. Dies ermöglicht eine einfache und saubere Handhabung dynamischer Arrays mit Objekten. Die Objekte werden manuell mit *delete* freigegeben, sobald sie den sichtbaren Bereich verlassen. Genau wie in den anderen Versionen wird in *drawCanvas()* über die Höhe und Breite iteriert und mit Hilfsfunktionen den Boden, den Spieler und die Hindernisse zu bestimmen. Die Haupt-Schleife wird anfangs alle 50ms durchlaufen und beschleunigt sich proportional zum Punktestand. Wenn die Funktion *checkCollision()* *true* zurückgibt, stoppt das Spiel und endet fehlerfrei. Die C++ -Version nutzt diverse Bibliotheken wie z.B *<iostream>* für z.B Standardausgaben oder *<vector>* für dynamische Arrays.

4.2.3 Javascript / Node.js

Die Javascript-Version ist strukturell ähnlich zur C++-Version. Auch hier werden Klassen und Objekte zur Darstellung und Organisation von Spieler und Hindernis genutzt. Die Spiel-Schleife ist asynchron, da mit `setTimeout()` auf den Delay gewartet werden muss, bevor ein neuer Frame gerendert wird. Über das `readline`-Modul werden Tastatureingaben erfasst. Im Gegensatz zur C++-Version wird erfolgt die Speicherverwaltung nicht manuell, sondern über eine automatische Garbage-Collection. In Javascript müssen bei Variablen auch keine Datentypen angegeben oder Pointer genutzt werden. Statt `std::vector` werden Arrays mit `filter()` für die Verwaltung der Hindernisse verwendet.

4.3 Messung und Vergleich

Die Ressourceneffizienz wurde unter folgenden Bedingungen gemessen: Jedes Spiel lief dreimal, jeweils 60 Sekunden, der Durchschnitt wird als Wert erfasst. Gemessen wurde hierbei die CPU- und RAM-Auslastung, die Ladezeit, die Kompilierungszeit sowie der durch die ausführbaren Datei belegte Speicherplatz. Bei Javascript wurde die reine Source-Code-Größe erfasst, da Node.js als Systemvoraussetzung gilt.

	x86_64 Assembly	C++	Javascript
Ø CPU-Auslastung	~12,26%	1,29%	19,77 %
Maximale CPU-Auslastung	17,4%	3,9%	31,5 %
Ø RAM- Auslastung	0.0 MB	~3,7333 MB	~62,3667 MB

Überraschenderweise zeigt die C++-Version die niedrigste durchschnittliche CPU-Auslastung (1,29%), gefolgt von Assembly (~12,26%) und Javascript(19,77%). Dies könnte sich durch die hohe Nutzung von CPU-Registern und kaum Nutzung von Arbeitsspeicher in Assembly erklären. Es könnte sich aber um fehlerhafte oder ineffiziente Funktionen im Source-Code des Assembly-Spiels handeln.

Die Arbeitsspeicher-Auslastung des Javascript-Spiels (~62,3667MB) ist am größten. Javascript benötigt den meisten Speicher durch die V8-Engine, Garba-

ge-Collection und JIT-Compiler. Dahingegen benötigt C++ lediglich ~3,73MB und Assembly verbraucht praktisch keinen RAM (0,0MB).

	x86_64 Assembly	C++	Javascript
Ladezeit	0.002s	0.241s	~0.58233s
Kompilierungszeit	~21.33ms	~615.667ms	/
Belegter Speicherplatz	14KB	18KB	6.7KB

Die Assembly-Version lädt nahezu sofort, da z.B keine Runtime initialisiert werden muss. C++ ist etwa 120x länger aufgrund von z.B Eingliederung von Bibliotheken. Javascript hat die längste Ladezeit durch den Start der V8-Engine.

Der Assembler hat eine deutlich schnellere Kompilierungszeit, da nahezu 1:1 in Maschinencode übersetzt wird. C++ hingegen wird zwar auch in Maschinencode übersetzt, jedoch führt der Compiler aufwendige Optimierungsprozesse durch, was den Kompilierungsprozess deutlich zeitintensiver macht. JavaScript wird interpretiert, daher entfällt die Kompilierungszeit.

Die ausführbare Assembly-Datei ist ein 4KB kleiner als die von entsprechende C++-Binary. Dies liegt an den zusätzlichen Bibliotheken die in C++ eingebunden werden. Der Source-Code von Javascript ist zwar deutlich kompakter, jedoch ist er ohne Node.js nicht ausführbar. Berücksichtigt man die Größe der Node.js-Runtime, relativiert sich der scheinbare Vorteil der kleinen JavaScript-Datei, da die tatsächliche Ausführungsumgebung wesentlich umfangreicher ist.

Insgesamt zeigt sich, dass sich meine anfängliche These nur teilweise bestätigt. Die Programmiersprachen, welche näher an der Maschinenebene operieren, erzielen auch in den meisten Kategorien die besten Ergebnisse hinsichtlich Ressourceneffizienz. Allerdings bedeutet technische Effizienz nicht unbedingt Effizienz in der Entwicklung. Rückblickend war die Entwicklung der Spiele mit hardwarenahen Programmiersprachen deutlich anspruchsvoller, komplexer und insgesamt zeitintensiver. Hier wird erneut deutlich, dass Ressourceneffizienz nicht der einzige relevante Faktor ist und dass man sie häufig zugunsten von Lesbarkeit, Wartbarkeit und Entwicklungsgeschwindigkeit abwägen muss.

C++ erwies sich in diesem Zusammenhang als geeigneter Mittelweg. Die Sprache bietet ein gutes Gleichgewicht von Hardwarenähe, struktureller Lesbarkeit und effizienter Programmierung. Dadurch ermöglicht sie eine vergleichsweise hohe Performance und eine praktikable Entwicklungsumgebung.

5 Fazit

Die Wahl der Programmiersprache hat einen messbaren, jedoch kontextabhängigen Einfluss auf die Ressourceneffizienz von Software. Anhand der Fallstudie *RollerCoaster Tycoon* wurde deutlich, dass hardwarenahe Programmierung in Assembly unter den Bedingungen der späten 1990er-Jahre essentiell war, um komplexe Simulationen flüssig darzustellen. Trotz isometrischer, pixeliger Grafiken war es ein voller Erfolg und läuft auch heute auf modernen Systemen problemlos, ohne veraltet zu wirken.

Das entstandene Minispiel besitzt auch nur eine simple Grafik, dennoch lassen sich Unterschiede zwischen den Programmiersprachen feststellen. Assembly war insgesamt am ressourceneffizientesten, gefolgt von C++ und Javascript. Die Unterschiede in der CPU-Auslastung und dem Speicherverbrauch waren statistisch nachweisbar, fielen jedoch bei einem so grafisch und rechnerisch simpel gehaltenen Projekt nicht menschlich wahrnehmbar aus. Als Nachteil der Ressourceneffizienz erwies sich die lange Entwicklungsdauer und schlechte Lesbarkeit und Skalierbarkeit, welche sie mit sich bringt.

In der Praxis sollte hardwarenahe Programmierung vor allem dort eingesetzt werden, wo maximale Ressourceneffizienz unverzichtbar ist. Für die überwiegende Mehrheit an Spielprojekten eignet sich eher beliebte, moderne Hochsprachen wie C++ oder Javascript. Entscheidend bleibt dabei stets die Qualität der Implementierung. Eine ineffizient geschriebene C++-Anwendung kann von einer optimierten Javascript-Lösung übertroffen werden.

Zusammenfassend bestätigt sich am Beispiel des simplen, aber unterhaltsamen Terminal-Spiels die Erkenntnis aus *RollerCoaster Tycoon*. Ressourceneffizienz ist nur teilweise abhängig von der Wahl der Programmiersprache und man kann mit gelungener Spielmechanik in jeder Sprache Spielspaß erzeugen, solange die Hardware es zulässt.

[2zCIng3JSOrMS1U6d6B9QHmnAxKd1aoFnxU](#), eingesehen 25.02.2026 02:16

Luber, Stefan und Jürgen Ehneß: *Was ist Speicherkapazität?*. Storage-insider.de, 21.01.2020, verfügbar unter: <https://www.storage-insider.de/was-ist-speicherkapazitaet-a-885599/>, eingesehen am 25.02.2025 00:50

Mozilla Foundation: *Javascript*. Developer.mozilla.org, verfügbar unter: <https://developer.mozilla.org/de/docs/Web/JavaScript>, eingesehen 25.02.2026 03:31

NumPy Developers: *What is NumPy?*. NumPy.org, verfügbar unter: <https://numpy.org/doc/stable/user/whatnumpy.html>, eingesehen 25.02.2026 01:43

Rauch, Gedeon: *Was sind höhere Programmiersprachen?*. Datacenter-inside.de, 11.03.2022, verfügbar unter: <https://www.datacenter-insider.de/was-sind-hoehere-programmiersprachen-a-35361189a0d1b8c91cb7b8f209abab10/>, eingesehen am 25.01.2026 20:07

Red Phoenix: *Interview with Chris Sawyer*. RCT Station Archive (archiviert): Internet Archive: Wayback Machine, 21.12.2013, verfügbar unter: <https://web.archive.org/web/20131221171913/http://www.rctstationarchive.com/interviews/chrissawyer.shtml>, eingesehen am 04.02.2026 21:45

Sawyer, Chris (Entw.): *RollerCoaster Tycoon* [Computerspiel]. Hasbro Interactive, 1999. Gespielt am 08.02.2026 12:13

Sawyer, Chris (Entw.): *Transport Tycoon* [Computerspiel]. MicroProse, 1994. Gespielt am 06.02.2026 13:12

Sawyer, Chris: ...about Chris Sawyer & Game Development. chrissawyergames.com, verfügbar unter: <https://www.chrissawyergames.com/faq3.htm>. eingesehen am 31.10.2025 15:20

TU Dresden, Professur Betriebssysteme: *Assembler Crash Course (x86_64)*. Os.inf.tu dresden.de, verfügbar unter: <https://os.inf.tu-dresden.de/Studium/OSC/SS2022/tasks/task4/assembler.php>, eingesehen am 25.02.2026 02:37

6.2 Abbildungsverzeichnis

Abbildung 4: u/Innadiated: *Innadiated's Pacific Pyramids*. Reddit.com, verfügbar über: https://www.reddit.com/r/rct/comments/1px0b9p/innadiateds_pacific_pyramids/, eingesehen am 25.02.2026 05:25

Abbildung 5: Chrome Dino Offline-Spiel (Original). Eigener Screenshot aus: Google Chrome Browser, erstellt am 25.02.2026 05:49

7 Anhang

Praxisteil, Messungen und Videos unter: <https://github.com/iveltier/facharbeit>

8 Eidesstattliche Erklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe, dass alle Stellen der Arbeit, die wörtlich oder sinngemäß aus anderen Quelle übernommen wurden, als solche kenntlich gemacht sind und dass die Arbeit in gleicher oder ähnlicher Form noch an keiner anderen Stelle vorgelegt wurde.

Unterschrift des Schülers

Stadt, Datum